

Содержание

Введение	3
Основная часть	4
1. РЕР 827 и tysql: запрос как тип	4
2. Методология двух дорожек	4
3. Оценка на PostgreSQL-оракуле	5
4. Границы применимости и вклад	5
Заключение	6
Список литературы	6

Введение

Актуальность. Статическая система типов Python не способна выразить типы, порождаемые метапрограммированием, хотя динамическое создание классов встроено в язык с самого начала. Библиотеки возвращают значения, точную форму которых система типов описать не может: `model_dump()` в Pydantic объявлен как `dict[str, Any]`, а конструкторы SQL-запросов возвращают нетипизированные строки. PEP 827 (*Type Manipulation*, статус Draft, Python 3.16) вводит средства манипуляции типами — комбинаторы, вычисляющие новые типы из существующих, — призванные закрыть этот пробел. Достаточны ли они на практике, оставалось неизвестным.

Цель — эмпирически оценить достаточность средств PEP 827, построив на них конструктор запросов к PostgreSQL (`tysql`), в котором тип результата статически выводится из самого запроса, и сверив выведенные типы с реальной базой данных.

Задачи: выделить типизируемое подмножество `S` запросов; реализовать `tysql`, представляющий схему и запросы как типы; разработать методологию «двух дорожек» (интерпретатор `typemap` и форк `шуру`) с честными негативными тестами; построить оценку на PostgreSQL-оракуле; очертить границы применимости и внести вклад в эталонный инструментарий PEP 827.

Объект — система типов Python и средства PEP 827; **предмет** — их достаточность для статического вывода типов результатов SQL-запросов.

Научная новизна: впервые в Python реализован и подтверждён статический вывод типа результата SQL-запроса из самого запроса — с верификацией против реального PostgreSQL и точной картой невыразимых конструкций.

Основная часть

1. PEP 827 и `tysql`: запрос как тип

PEP 827 предлагает комбинаторы уровня типов — `Attrs[T]`, `Iter`, `Member`, `NewTypedDict`, `IsAssignable` и другие, — строящие новые типы из существующих без нового синтаксиса. Схема базы данных в `tysql` описывается классами-таблицами, а каждый оператор — типом, например `Select[User, Cols[...]]`; выражение `ResultOf[stmt]` вычисляется в `TypedDict` с точными именами и типами столбцов. Реализованы `CREATE TABLE`, `INSERT ... RETURNING`, проекция с псевдонимами, `WHERE` с выводом параметров, `INNER JOIN`, агрегат `Count` с `GROUP BY/ORDER BY` и рендеринг в текст PostgreSQL. Ключевая сложность — двойственность аннотаций: одна и та же аннотация является и статической подсказкой, и (с Python 3.14) лениво вычисляемым выражением; решение — держать одну спецификацию сразу на обеих дорожках и требовать их согласия.

2. Методология двух дорожек

Каждая возможность проверяется одновременно интерпретатором `typer` (через `pytest` с интроспекцией полученных классов) и статическим форком `myru`. Негативные проверки сделаны честными: каждый `# type: ignore[code]` под флагом `--warn-unused-ignores` становится проверяемым отрицательным утверждением — как только ошибка перестаёт выдаваться, подавление оказывается неиспользуемым и сборка краснеет. Расхождения дорожек фиксируются метками `xfail(strict=True)`. Именно парность дорожек позволила выявить тонкие дефекты как в самом `tysql`, так и в эталонном инструментарии.

3. Оценка на PostgreSQL-опракуле

Авторитетом, которым сам `tysql` не является, взят реальный сервер PostgreSQL: `cursor.description` сообщает истинные имена и типы столбцов результата. Для каждого оператора сценарий рендерит SQL и исполняет его на PostgreSQL 17, вычисляет `ResultOf[stmt]` через `typemap` и сверяет обе стороны по именам, порядку и типам. На подмножестве **S** все классы запросов совпали с PostgreSQL точно: полная строка, проекция, псевдоним, WHERE с параметрами, INNER JOIN и агрегат `Count + GROUP BY`. Проверка автоматизирована: она оформлена как интеграционный тест на `testcontainers` (поднимающий PostgreSQL 17 в контейнере) и выполняется отдельной задачей непрерывной интеграции.

4. Границы применимости и вклад

За границей **S** ограничения выстраиваются в «лестницу точности»: *огрубление* — `Count` типизирован как `int`, тогда как PostgreSQL возвращает `bigint`; *невыразимость* — нет комбинатора для `SUM/AVG`, дублирующиеся имена столбцов не помещаются в `TypedDict`, а `nullability` внешнего соединения не сообщается протоколом; *несоответствие* — сравнение `Eq` над несравнимыми операндами принимается `tysql`, но отвергается PostgreSQL. Работа принесла прямой вклад в эталонный инструментарий PEP 827: одно исправление принято (PR #117), ещё одно предложено (PR #122), обнаруженный дефект оформлен как issue (#123). Наконец, `tysql` поставляет CLI-валидатор (`tysql check`), опубликован на PyPI и снабжён браузерным playground.

Заключение

На подмножестве **S** — `CREATE TABLE`, `INSERT ... RETURNING`, проекция с псевдонимами, `WHERE` с выводом параметров и `INNER JOIN` с `Count`, `GROUP BY`, `ORDER BY` — средств PEP 827 достаточно: `tysql` выводит точный тип результата каждого запроса, что подтверждено сверкой столбец за столбцом с реальным PostgreSQL. За пределами **S** границы названы поимённо: сравнение значений несоудно, `SUM`/`AVG` и nullability внешних соединений невыразимы, дублирующиеся имена разрушают модель записи; часть «стен» относится к прототипу проверяющего, а не к самому предложению. Практические результаты — рабочий конструктор запросов уровня типов и его CLI-валидатор, методология двух дорожек и воспроизводимая оценка на PostgreSQL-оракуле — дают конкретное свидетельство того, что вычисления на уровне типов в Python можно превратить в спроектированную и переносимую поверхность, и повышают шансы PEP 827 на принятие.

Список литературы

1. PEP 827 — Type Manipulation. URL: <https://peps.python.org/pep-0827/>
2. PEP 484 — Type Hints. URL: <https://peps.python.org/pep-0484/>
3. PEP 649 — Deferred Evaluation of Annotations. URL: <https://peps.python.org/pep-0649/>
4. PEP 692 — Using TypedDict for More Precise `**kwargs` Typing. URL: <https://peps.python.org/pep-0692/>
5. `tysql` — конструктор запросов к PostgreSQL уровня типов. URL: <https://github.com/iliyasone/tysql>
6. `python-typemap` — интерпретатор PEP 827 времени выполнения (форк). URL: <https://github.com/iliyasone/python-typemap>
7. `mypy-typemap` — форк `mypy`, вычисляющий комбинаторы PEP 827. URL: <https://github.com/iliyasone/mypy-typemap>